

國立陽明交通大學

資訊安全研究所

碩士論文

Institute of Computer and Communications Security

National Yang Ming Chiao Tung University

Master's Thesis

Engage-Agent：基於 MITRE Engage 的雙層 LLM Agent

交戰式蜜網系統

Engage-Agent: A 2-Layer MITRE Engage-driven LLM

Agent in a Honeynet

研 究 生：王世揚 (Wang, Shih-Yang)

指導教授：林盈達 (Lin, Ying-Dar)

中華民國 一一五年八月

August 2026

Engage-Agent：基於 MITRE Engage 的雙層
LLM Agent 交戰式蜜網系統
Engage-Agent: A 2-Layer MITRE Engage-driven
LLM Agent in a Honeynet

研 究 生：王世揚

Student：Shih-Yang Wang

指導教授：林盈達 博士

Advisor：Dr. Ying-Dar Lin

國立陽明交通大學

資訊安全研究所

碩士論文

A Thesis

Submitted to Institute of Computer and Communications Security

College of Computer Science

National Yang Ming Chiao Tung University

in Partial Fulfillment of the Requirements

for the Degree of

Master of Science

in

Computer Science

August 2026

Taiwan, Republic of China

中華民國 一一五年八月

誌 謝

謝天謝地

王世揚 於

國立陽明交通大學 資訊安全研究所

中華民國 一一五年八月



Engage-Agent：基於 MITRE Engage 的雙層 LLM Agent 交戰式蜜網系統

學生：王世揚

指導教授：林盈達 博士

國立陽明交通大學
資訊安全研究所

摘 要

中文摘要就從這邊開始寫.

關鍵字：中文, 摘要, 關鍵詞, 5-7 個, 不要多, 也不要少

Engage-Agent: A 2-Layer MITRE Engage-driven LLM Agent in a Honeynet

Student : Shih-Yang Wang

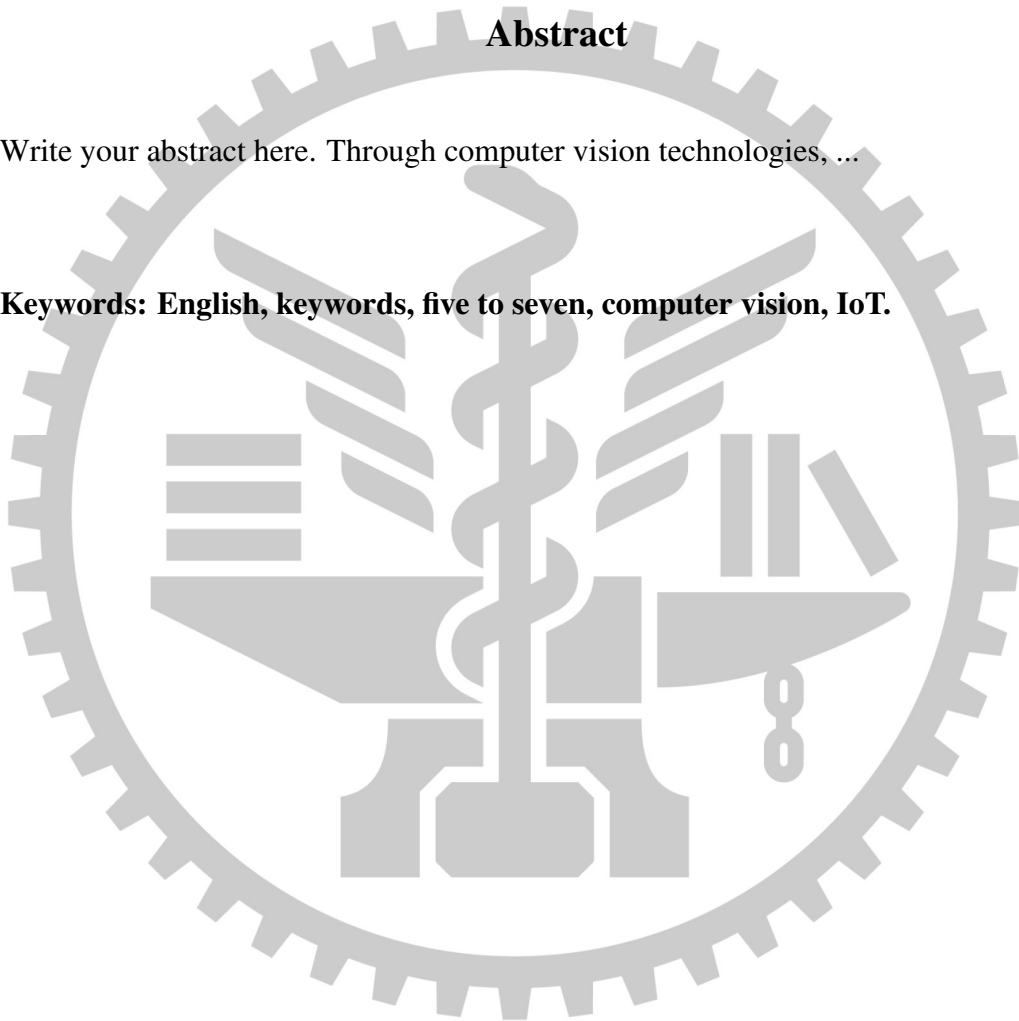
Advisor: Dr. Ying-Dar Lin

Institute of Computer and Communications Security
National Yang Ming Chiao Tung University

Abstract

Write your abstract here. Through computer vision technologies, ...

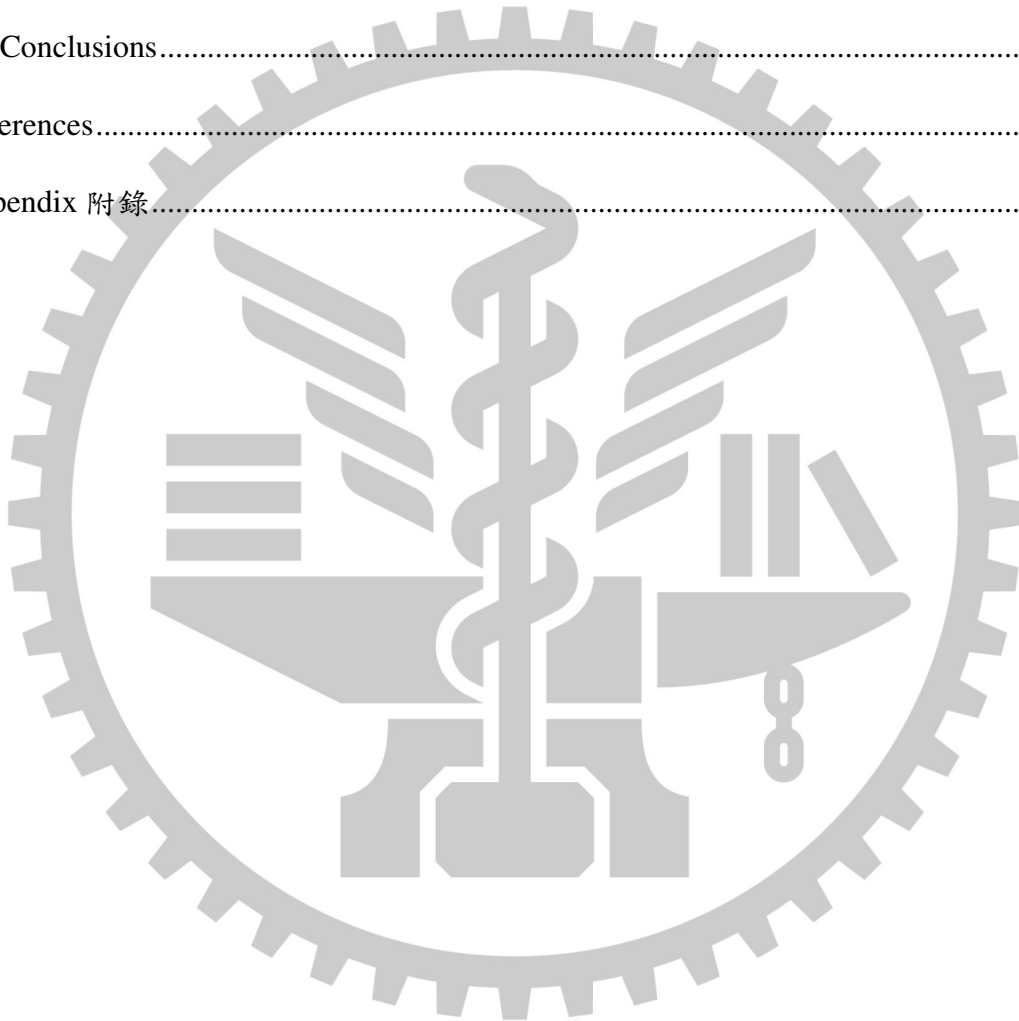
Keywords: English, keywords, five to seven, computer vision, IoT.



Contents

Chinese Abstract	i
English Abstract	ii
Contents	iii
List of Figures	v
List of Tables	vi
1 Introduction	1
2 Background and Related Work	3
2.1 Honeypot and Honeynet	3
2.2 MITRE ATT&CK and MITRE Engage	3
2.3 LLM Agent and MCP	4
2.4 Active Directory	5
3 Problem Formulation	9
3.1 Notations	9
3.2 Problem Statement	10
4 Solution Approach	12
4.1 Solution Overview	12
4.2 Strategist LLM and Tool Selection	14
4.3 Planner LLM and MCP Toolkit	15
4.4 AD Domain Environment Design	18

5	Implementation	21
5.1	System Architecture.....	21
5.2	Dataset Generation and Fine-Tuning	22
5.3	MCP Toolkit Generation and Execution	25
5.4	Attacker Interaction Simulation	26
6	Experiment Result.....	28
7	Conclusions.....	30
	References.....	32
	Appendix 附錄.....	34



List of Figures

Figure 1	MCP Server	5
Figure 2	Active Directory	6
Figure 3	Solution overview	13
Figure 4	Role of the Strategist LLM in attacker-intent inference and tool selection .	15
Figure 5	Tool-specification-guided plan generation and script rendering by the Planner LLM and MCP toolkit	18
Figure 6	AD-domain-based honeynet environment with role-specific hosts and services	19
Figure 7	Runtime architecture of the implemented Engage-Agent system.	22
Figure 8	Offline dataset generation pipeline for Strategist supervision construction and Planner fine-tuning.	23
Figure 9	Semi-automated generation and validation process for MCP tools.	25
Figure 10	Excel2LaTeX.	29
Figure 11	Excel2LaTeX.	31

List of Tables

Table 1	Survey on recent research in adaptive honeypots and honeynets	7
Table 2	Notations	9
Table 3	Summary of MCP tool groups and their alignment with selected MITRE Engage activities	17
Table 4	Dataset construction scale in the offline supervision pipeline	23
Table 5	Planner supervised fine-tuning configuration	24
Table 6	This is a table.	28
Table 7	Comparison on model efficiency and model size.	28
Table 8	This is a table.	30
Table 9	Comparison on model efficiency and model size.	30

Chapter 1. Introduction

Modern enterprise attacks, such as lateral movement, credential reuse, and Active Directory compromise, are inherently multi-stage and span across multiple hosts, requiring realistic environments to study attacker behavior progression. Honeypots, as security deception systems designed to attract attackers and collect behavioral data, have been widely used in controlled environments for this purpose. However, traditional honeypots typically simulate only a single host, making them insufficient for capturing multi-stage, cross-host attack progression. As a result, they suffer from limited attack depth and incomplete observation of attacker behavior progression, restricting their effectiveness in analyzing complex enterprise-scale attacks.

To address these limitations, honeynet environments have been proposed, where multiple interconnected honeypots form a multi-host network that better reflects real enterprise infrastructures. Such environments enable the observation of cross-host attack behaviors and improve realism compared to single-host honeypots. Nevertheless, existing honeynet research, such as HoneyFactory[1] and ADLAH[2], primarily focuses on environment construction and dynamic deployment. These systems largely operate in a passive observation mode, lacking mechanisms for active interaction with attackers. Consequently, they are unable to influence attacker decisions or guide attack progression, which limits attack depth, reduces behavioral diversity, and weakens engagement effectiveness. This reveals a fundamental gap between environment realism and adaptive interaction capability in current honeynet systems.

In parallel, adaptive honeypot research has introduced dynamic interaction capabilities using reinforcement learning (RL) and large language model (LLM)-based decision mechanisms. Systems such as QRASSH[3] and HoneyGPT[4] demonstrate the ability to generate engagement strategies and interact with attackers in real time. However, these approaches are designed for single-host settings and cannot be directly extended to honeynet environments. A direct combination of multiple independent adaptive honeypots results in independent decision making without global coordination, failing to manage cross-host attack progression. Extending

adaptive interaction to honeynets introduces significant challenges, including a large decision space, complex multi-host system states, and the need for cross-host consistency. More importantly, there is currently no unified framework that enables coordinated, environment-level, multi-host engagement. Therefore, a key challenge is to design an adaptive honeynet system that simultaneously achieves enterprise-level realism and globally coordinated attacker interaction.

To address this challenge, this paper proposes an adaptive honeynet engagement framework that integrates global decision making with environment-level deception. Instead of generating simulated reply content, the proposed approach dynamically modifies the underlying system environment across multiple hosts, ensuring that attacker interactions emerge naturally from actual system state changes. The framework employs an LLM-based decision agent that analyzes attacker commands, infers attacker intent, and generates coordinated engagement strategies. These strategies are executed through environment-level actions across multiple hosts, enabling guided attack progression. Through this design, the system achieves deeper attack exploration, higher behavioral diversity, and realistic multi-host interactions.

The main contributions of this work are summarized as follows:

- We propose an adaptive honeynet engagement framework that bridges the gap between environment realism and adaptive attacker interaction.
- We introduce an environment-driven deception approach that performs system-level modifications, enabling attacker interactions to emerge from real system behavior rather than simulated replies.
- We design an MITRE Engage-aligned engagement strategy that maps attacker intent to structured deception activities, bridging the gap between high-level adversary engagement goals and executable environment-level actions.

The remainder of this paper is organized as follows. Chapter 2 presents the background and related work. Chapter 3 introduces the problem formulation. Chapter 4 describes the proposed solution approach. Chapter 5 details the system implementation. Chapter 6 presents experimental results. Finally, Chapter 7 concludes the paper and discusses future work.

Chapter 2. Background and Related Work

2.1 Honeypot and HoneyNet

Honeypots are security deception systems that simulate a single host, while honeynets extend this concept by emulating an entire network environment. Both aim to safely attract attackers and analyze their behaviors and techniques. In practice, open-source tools such as Cowrie enable the deployment of high-interaction static honeypots, and platforms like T-Pot allow multiple honeypots to be deployed together as a static honeynet. Commercial deception technologies, including Zscaler Deception and Proofpoint Identity Threat Defense, deploy decoys within real enterprise environments to lure attackers. Detected malicious activities are then redirected to controlled sandbox environments for observation.

However, existing solutions primarily focus on capturing attacks rather than actively engaging with attackers. They lack mechanisms to dynamically interact with and guide attackers toward deeper and more diverse attack behaviors, which limits their value for comprehensive security analysis. This work aims to fill this gap by enabling active and adaptive engagement within honeynet environments.

2.2 MITRE ATT&CK and MITRE Engage

MITRE ATT&CK[5] is a widely used knowledge base for characterizing adversary tactics, techniques, and procedures. It is primarily designed to support attack analysis, threat intelligence, and detection engineering by describing attacker behavior in a structured form. However, ATT&CK focuses on modeling attacker actions and does not explicitly provide guidance on how defenders should respond during adversary engagement. Therefore, although ATT&CK is useful for interpreting attack behavior, it is not sufficient for guiding adaptive deception in this work.

To address this limitation, MITRE proposed MITRE Engage[6], a framework specifically designed for active defense, deception, and adversary engagement. MITRE Engage has been designed with the explicit goal of enabling defenders to interact with attackers more effectively and collect richer behavioral data. The framework organizes defender knowledge into three layers: goals, approaches, and activities, thereby linking high-level engagement objectives to concrete defensive actions. Compared with ATT&CK, which emphasizes attack representation, Engage is more suitable for this paper because it provides a clearer basis for selecting and executing engagement actions in a honeynet.

Accordingly, this work adopts MITRE Engage as the conceptual foundation for engagement strategy design. The proposed system uses this framework to guide the selection and execution of environment-level deception activities that dynamically influence attacker behavior within the honeynet.

2.3 LLM Agent and MCP

LLM-based agents are AI decision systems driven by large language models, capable of reasoning, planning, and invoking external tools to perform actions. They typically operate in a perception–reasoning–action loop, where the agent analyzes environmental context, generates decisions, and executes actions through integrated tools.

In the context of this work, directly generating executable Ansible scripts using LLMs is unreliable, as free-form generation often leads to invalid syntax, inconsistent configurations, and low execution success rates. Instead of allowing the LLM to generate scripts from scratch, this paper adopts a tool-based approach, where environment modifications are performed through a predefined set of executable tools. Each tool encapsulates a verified Ansible template and exposes a structured interface, enabling the LLM to generate only the necessary parameters rather than full scripts. This design significantly improves execution stability and ensures that all actions are grounded in valid system operations.

To support structured tool interaction, the Model Context Protocol (MCP) provides a standardized interface that connects LLM agents with tools and environments. As illustrated in Figure 1, MCP abstracts multiple external services into a unified interface, allowing the LLM to interact with diverse systems through a single API layer. In this work, MCP is adopted as

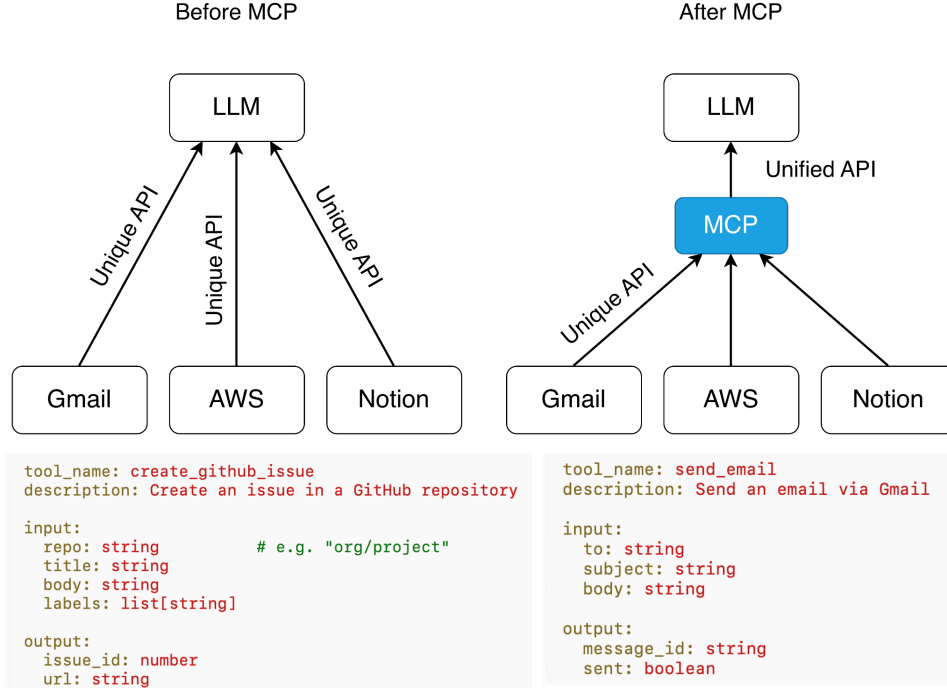


Figure 1: MCP Server

a practical implementation choice for defining tool interfaces and managing execution, rather than a strict requirement of the system design. The key contribution lies in the tool-based execution paradigm, while MCP serves as a convenient and widely-used framework to realize this interaction model.

2.4 Active Directory

Active Directory (AD) is a directory service developed by Microsoft for centralized management of users, hosts, authentication, authorization, and security policies in Windows-based enterprise environments. In real organizations, AD commonly serves as the core infrastructure that connects accounts, endpoints, servers, and administrative control within a domain, as shown in Figure 2. Because of this central role, AD is not only widely used in enterprise networks but also strongly shapes how attackers move, escalate privileges, and access resources in practice.

This importance also makes AD a high-value attack target. Once attackers compromise an AD environment, they can often expand access across multiple hosts, abuse trust relationships, and gain broad control over enterprise resources. As a result, many real-world attack techniques

are closely tied to AD, including credential abuse, lateral movement, and privilege escalation. Representative examples such as Pass-the-Hash and Kerberoasting further show that AD is not merely a background service, but a major operational surface in enterprise attacks.

For this reason, this work adopts AD as the foundation of the honeynet environment. Since adaptive deception in this work is evaluated through multi-step enterprise attack behavior, the experimental environment must expose the same identity structure, trust relationships, and privilege dependencies that attackers encounter in real networks. An AD-based honeynet therefore provides a realistic and representative basis for evaluating whether the proposed system can influence attacker progression under practical enterprise attack conditions.

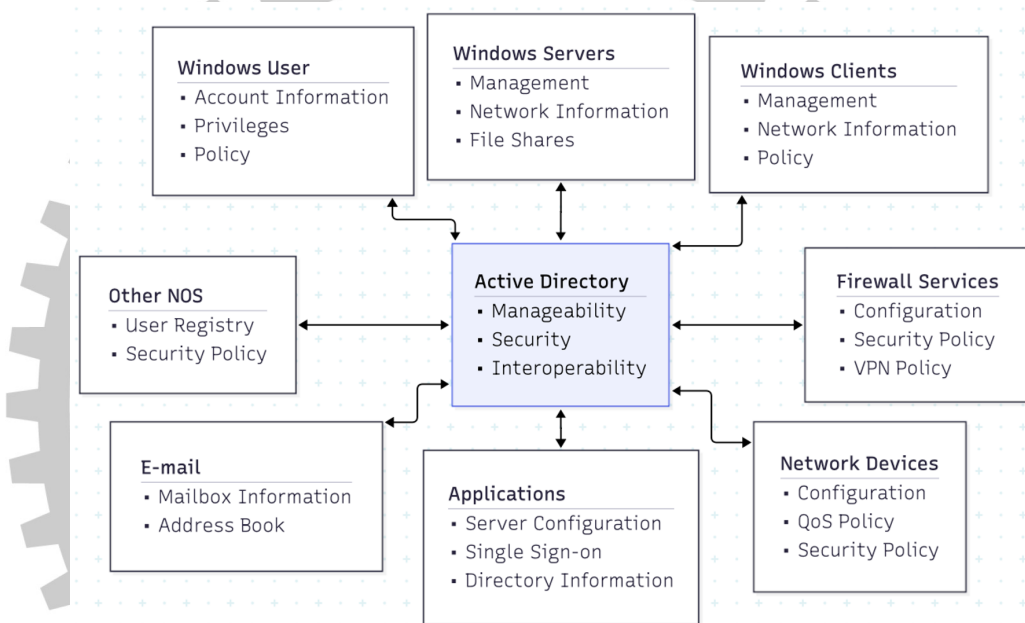


Figure 2: Active Directory

Existing research on adaptive deception systems can be broadly categorized into two directions: adaptive honeypots and adaptive honeynets. These two lines of work focus on different aspects of attacker interaction and environment design.

Adaptive honeypot research primarily focuses on dynamically engaging with attackers at the single-host level. Representative systems such as QRASSH[3], HoneyGPT[4], and Senior[7] introduce reinforcement learning (RL) or large language model (LLM)-based decision mechanisms to model attacker commands and generate responses. These systems typically operate at the command or reply level, selecting actions from a limited action space to influence attacker behavior. Their primary objective is to prolong attacker engagement and collect richer behav-

Table 1: Survey on recent research in adaptive honeypots and honeynets

Category	Paper	Real	AD	Objective			Method	Action Design
				Depth	Realism	Diversity		
Honeypot	QRASSH [3]	–	–	✓	–	–	RL	Response manipulation
	HoneyGPT [4]	–	–	✓	✓	✓	LLM	Response generation
	Senior [7]	✓	✓	✓	✓	✓	RL	Response manipulation
Honeynet	HoneyFactory [1]	✓	–	–	✓	–	Rule-based	Honeynet generation
	ADLAH [2]	–	–	–	✓	–	RL	Honeypot deployment
	HoneyICS [8]	✓	–	–	✓	–	Rule-based	ICS network emulation
	Mirage [9]	✓	✓	–	–	–	Rule-based	File-system modification
	Ours	✓	✓	✓	✓	✓	LLM	Mitre Engage-guided environment modification

ioral data. However, due to their single-host nature, these systems are limited in their ability to capture multi-stage attacks involving lateral movement and cross-host interactions.

In contrast, adaptive honeynet research focuses on constructing realistic multi-host environments that emulate enterprise infrastructures. Systems such as HoneyFactory[1], ADLAH[2], HoneyICS[8], and Mirage[9] represent this direction. HoneyFactory[1] dynamically generates honeynet environments with enterprise-like topologies to capture multi-stage attacks across multiple hosts. ADLAH[2] applies reinforcement learning to optimize honeypot placement and redirects anomalous connections to backend systems such as Cowrie, improving attack capture efficiency. HoneyICS[8] targets industrial control system (ICS) environments by simulating physical processes and IoT devices to capture domain-specific attacks. Mirage[9] adopts an environment-level deception approach by modifying file system artifacts using rule-based strategies to influence attacker perception and behavior. To systematically compare existing approaches, Table 1 summarizes representative adaptive honeypot and honeynet systems in terms of engagement capability, environment realism, and behavioral coverage. While these systems improve environmental realism and enable multi-stage attack observation, they generally rely on static or rule-based strategies and lack mechanisms for adaptive, semantic-level interaction with attackers.

As shown in Figure 1, adaptive honeypot systems emphasize interaction intelligence, leveraging RL or LLM-based decision mechanisms to dynamically respond to attackers. However,

these systems primarily operate at the reply level and are limited to single-host environments, resulting in restricted support for lateral movement and reduced enterprise realism. Consequently, they cannot effectively model complex multi-stage attack scenarios.

In contrast, honeynet systems focus on environment realism by constructing multi-host infrastructures and supporting environment-level artifact manipulation. These systems enable the observation of cross-host attack behaviors and improve realism compared to single-host honeypots. However, as indicated in Table 1, their strategies are typically static or rule-based, focusing on deployment optimization or attacker disruption rather than adaptive engagement driven by semantic reasoning. As a result, their ability to actively influence attacker behavior and guide attack progression remains limited.

These observations reveal a fundamental gap between interaction intelligence and environment realism. Existing approaches do not provide a unified framework that integrates dynamic attacker engagement with enterprise-scale environments. In particular, prior work lacks the combination of semantic-aware reasoning, coordinated multi-host decision making, and environment-level execution. This gap motivates the need for an adaptive honeynet framework that combines LLM-based decision making with environment-level manipulation to enable realistic and guided multi-stage attack progression. To address this gap, this paper proposes Engage-Agent, which is detailed in the following chapters.

Chapter 3. Problem Formulation

This chapter formulates adaptive deception in a honeynet as a sequential, multi-host decision and deployment problem. Unlike traditional adaptive honeypots that mainly manipulate reply content on a single host, our setting requires the defender to reason over attacker behavior across multiple interaction steps and coordinate executable environment-level modifications over multiple hosts in a persistent enterprise-like honeynet. The goal is not merely to generate plausible responses, but to steer the attacker toward deeper and more diverse attack progression through realistic system-state manipulation while preserving execution reliability and real-time responsiveness.

3.1 Notations

Table 2: Notations

Categories	Notation	Description
Interaction & Session	$C_i, i \in \{1, \dots, N_C\}$	The i -th command issued by the attacker
	$\mathcal{C}_i = (C_{i-k+1}, \dots, C_i)$	Recent attack command context observed at interaction step i
	R_i	Response observed by the attacker after executing C_i
	N_C	Number of commands in one engagement session
	k	command context window size
Honeynet & Environment	$\mathcal{H} = \{H_1, \dots, H_{N_H}\}$	Set of hosts in the honeynet
	$H_j, j \in \{1, \dots, N_H\}$	A honeynet host (DC, web, mail, client, etc.)
	N_H	Number of computer host in the honeynet
LLM Agent & Planning	a_i	Engagement strategy selected by the Strategist LLM at interaction step i
	T_i	Subset of MCP tools selected by the Strategist LLM at interaction step i
	$a_i, T_i = f_{\text{strat}}(\mathcal{C}_i, T_i)$	Maps command context and available tools to strategy and selected tools
	$P = (p_1, p_2, \dots, p_{N_H}) = f_{\text{plan}}(a_i, T_i, T_s)$	Generated execution plans by the Planner LLM, $p_j = \emptyset$ if no action is assigned to host H_j
MCP & Execution	T_l	set of available MCP tools
	T_s	set of MCP tool specifications
	$S = (s_1, s_2, \dots, s_{N_H}) = \text{Render}(P)$	Executable scripts rendered from plans P
	$E = (e_1, e_2, \dots, e_{N_H}) = \text{Exec}(S)$	Execution results of scripts S

The main notations used in this chapter are summarized in Table 2. These symbols describe the problem from four aspects: attacker–honeynet interaction, the multi-host honeynet

environment, the two-layer LLM decision process, and MCP-based execution.

Specifically, C_i , $\mathcal{C}_i$, R_i , and N_C are used to represent the sequential interaction between the attacker and the honeynet. $\mathcal{H} = \{H_1, \dots, H_{N_H}\}$ denotes the set of honeynet hosts. Given the recent command context $\mathcal{C}_i$, the Strategist LLM produces a high-level strategy action and selected tools, and the Planner LLM further generates host-level plans $P = (p_1, \dots, p_{N_H})$. These plans are then rendered into executable scripts S and produce execution results E . Together, these notations formalize the end-to-end process from attacker interaction to environment-level deception execution.

3.2 Problem Statement

We formulate adaptive deception in a honeynet as a sequential decision problem over a persistent multi-host environment. At each interaction step i , the defender observes the recent attacker command context $\mathcal{C}_i$ and must determine how to modify the honeynet so that the attacker perceives realistic and coherent system feedback that steers attack progression toward deeper exploration.

The problem can be formulated as follows:

- **Input**

- $\mathcal{C}_i$: Recent attack command context observed at interaction step i

- **Output**

- R_i : Attacker-observed response induced by the execution result E of the generated plan P at step i

- **Objective**

- Maximize attack stage progression: $\max_{1 \leq i \leq N_c} s_i$, where $s_i \in \{1, 2, \dots, 7\}$ denotes the attack stage reached at step i

- **Constraints**

- Execution reliability: $\frac{\sum_{i=1}^{N_H} \mathbf{1}(e_i = \text{success})}{N_H} > 0.9$

- Real-time response constraints: $t_{strategist} + t_{planner} + t_{execution} \leq 90 \text{ (sec)}$

This problem introduces several fundamental challenges in applying adaptive deception to a persistent multi-host honeynet. First, there is an inherent tension between semantic reasoning and execution stability. Although LLMs provide strong contextual understanding and can infer attacker intent from command sequences, unconstrained free-form generation may produce outputs that are ambiguous, inconsistent, or difficult to execute reliably in real systems. As a result, the system must translate high-level semantic reasoning into constrained, tool-based actions that remain operationally stable. Second, the problem is complicated by multi-host coordination. Unlike single-host deception settings, an engagement strategy in a honeynet must affect multiple hosts in a coherent manner, while preserving cross-host and cross-step state consistency over time. Actions applied to one host may influence the plausibility and behavior of others, making coordinated planning substantially more difficult. Therefore, the core challenge is not only to generate deceptive actions, but to balance deception effectiveness, enterprise realism, and execution stability within a persistent multi-host honeynet.

Chapter 4. Solution Approach

This chapter presents the proposed solution to the adaptive honeynet engagement problem formulated in Chapter 3. The goal is to build an LLM-based cybersecurity framework that operates on a real-environment honeynet, monitors attacker behavior, and modifies the underlying environment to deceive the attacker and guide subsequent attack progression. Unlike traditional adaptive honeypots, which mainly manipulate replies on a single host, the proposed system is designed for a persistent multi-host setting in which attacker-visible responses emerge from actual system behavior after coordinated environment-level modifications.

4.1 Solution Overview

Figure 3 illustrates the overall workflow of the proposed adaptive honeynet engagement framework. The system consists of five main components: the attacker, the SSH proxy, the LLM agent, the defender MCP toolkit, and the AD domain environment. The attacker interacts with the honeynet through the SSH entry point, while the proxy serves as the interception point that captures commands and maintains recent command context. This context is then forwarded to the LLM agent, which is composed of a Strategist LLM and a Planner LLM.

As shown in Figure 3, at interaction step i , the recent attacker commands form the command context C_i . The Strategist LLM takes C_i as input, infers attacker intent, determines the engagement direction, and selects suitable tools from the available tool set T_l . The resulting strategy a_i and selected tools T_i are then passed to the Planner LLM, which converts them into an executable plan P by following the tool specifications T_s defined in the defender MCP toolkit. The MCP toolkit further renders P into executable scripts S and executes them in the AD domain environment across one or more hosts. The attacker subsequently observes response R_i , which is produced by the modified real environment rather than by directly fabricated replies. In this way, the system implements the Chapter 3 formulation as a sequential mapping from command context to strategy and tool selection, then to executable planning and script rendering, and

finally to attacker-observed response.

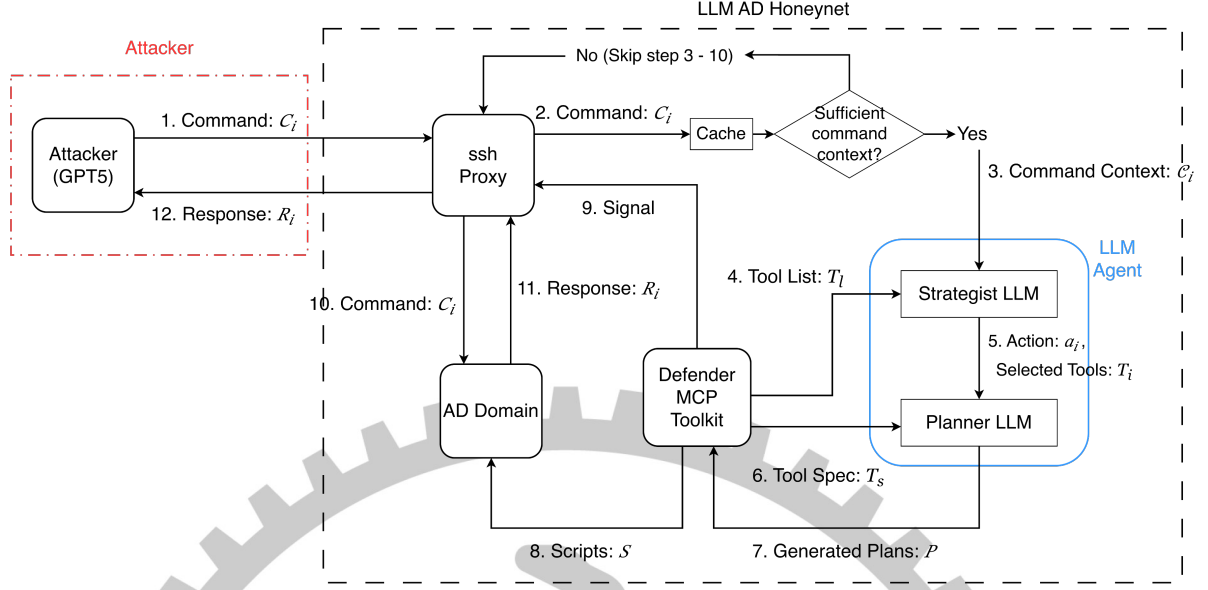


Figure 3: Solution overview

This design increases attack depth through the coordinated roles of the Strategist, the Planner, and real environment modification. The Strategist analyzes recent command context rather than isolated commands, allowing the system to infer the attacker’s current objective and choose engagement actions that match the current attack direction. The Planner and MCP toolkit then turn this high-level decision into dynamic but reliable modifications of the real honeynet, such as exposing files, accounts, credentials, services, or cross-host clues that are relevant to the attacker’s next step. As a result, the framework does not merely produce plausible replies, but continuously reshapes the environment to present believable opportunities that keep the attacker engaged and encourage progression toward later attack stages. By grounding later observations in actual system-state changes, this design achieves both persistent consistency and deeper attack progression.

Another key design choice is the use of two LLMs under practical local-LLM constraints. In an interactive honeynet, the agent must perform repeated reasoning with low operational cost and stable latency, making local deployment more practical than relying on a large remote model for every step. Under this constraint, a single-model design is too broad: one model would need to simultaneously understand attacker behavior, decide engagement direction, select tools, and prepare deployment-ready outputs. The proposed framework therefore separates these roles into Strategist LLM and Planner LLM. The Strategist focuses on contextual attacker

understanding and high-level engagement decision, while the Planner focuses on stable generation of execution-ready tool parameters. This separation makes each model address a narrower task, reducing inference complexity and improving controllability under local deployment constraints.

4.2 Strategist LLM and Tool Selection

Effective adaptive honeynet engagement requires the defender to infer the attacker’s current objective before deciding the next engagement action. This intent cannot be reliably determined from a single observed command, because the attacker’s goal is usually revealed by several recent commands taken together. As a result, immediate reaction to one command can easily misinterpret the attacker’s current direction. This also makes conventional reinforcement-learning-style state abstraction less suitable for the present problem, since the defender must first compress multi-command behavior into a fixed handcrafted state before any action can be selected. In a real honeynet, however, attacker behavior is open-ended, context-dependent, and semantically rich, making such manual state design difficult and brittle. The proposed framework therefore uses an LLM at this stage, allowing the system to reason directly over recent command context and infer what the attacker is currently trying to achieve before deciding how to engage.

As illustrated in Figure 4, the Strategist LLM takes recent command context C_i and the available MCP tool set as input, and converts them into a structured high-level engagement decision. It first infers attacker intention, so that the system can identify the attacker’s current objective rather than merely react to surface-level commands. It then generates engagement strategy a_i , which specifies the defender-side direction for the next step. Finally, it selects tools T_i from the available tool set, binding the strategy to a concrete subset of defender capabilities while keeping the decision separate from later deployment details. The output of this stage is therefore not a direct environment modification, but a structured decision consisting of attacker intention, engagement strategy, and selected tools.

Assigning tool selection to the Strategist keeps the choice of defender capabilities tightly coupled with attacker understanding. If this responsibility were deferred to the Planner stage, the high-level engagement decision would be split across two modules, weakening the connec-

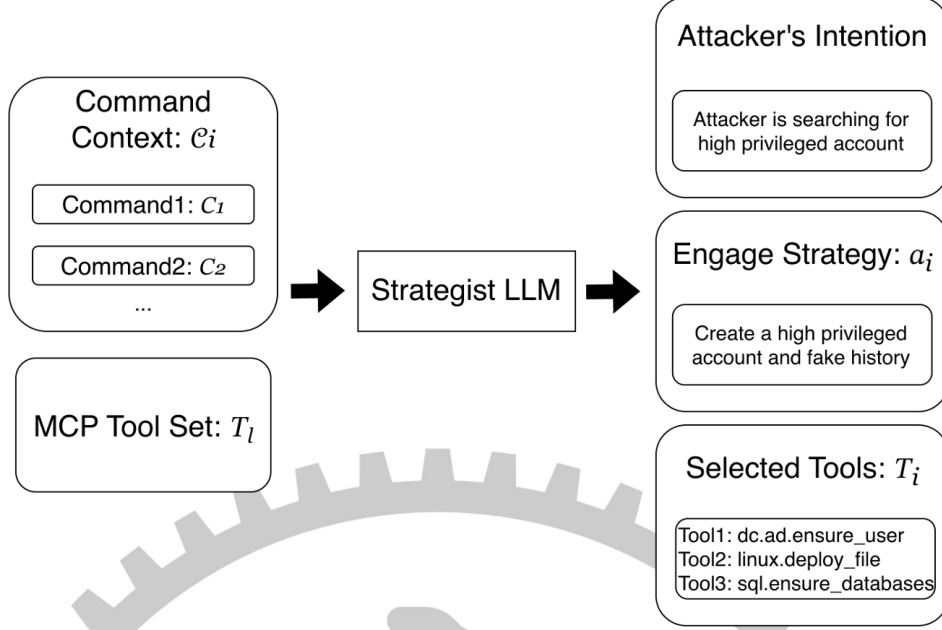


Figure 4: Role of the Strategist LLM in attacker-intent inference and tool selection

tion between attacker interpretation and defender response. At the same time, the Strategist stops at selecting tools and does not directly operate them or generate deployment scripts. Direct execution at this stage would mix attacker-oriented reasoning with host-specific deployment details, increase the burden of the first reasoning stage, and make the overall pipeline harder to control. The Strategist therefore decides which defender capabilities should be used, while the next stage determines how those capabilities should be instantiated in the environment. Because this stage primarily requires flexible semantic interpretation rather than highly regular structured output, prompt engineering is more appropriate than fine-tuning. Fine-tuning a small model on a fixed pattern of attacker contexts would constrain the Strategist’s ability to generalize across diverse attack scenarios, which is the opposite of what this stage requires.

4.3 Planner LLM and MCP Toolkit

After the Strategist determines the engagement strategy and selects tools for the next step, the defender must decide how these choices should be realized in the honeynet. Prior adaptive deception systems[3][4][10][7] often realize defender decisions by directly modifying or generating attacker-visible replies. This approach is flexible at the response level and can quickly create the desired attacker perception. However, in a honeynet environment, reply-level ma-

nipulation is fundamentally limited because later attacker actions may verify files, accounts, services, permissions, or cross-host relationships that were never actually changed. This inconsistency is especially problematic in a persistent multi-host setting, where the attacker can revisit earlier observations and validate them through follow-up commands. For this reason, the proposed framework chooses environment modification rather than direct reply fabrication. By changing the real environment, the system preserves cross-step consistency while still indirectly shaping the replies that the attacker later observes.

Choosing environment modification introduces a different requirement: the deployment method must remain dynamic enough to adapt to different attacker contexts while also remaining reliable enough to execute correctly in the real honeynet. Fixed scripts provide stable execution, but their adaptability is limited because one predefined script cannot cover the variety of attacker goals and contexts encountered across sessions. Direct script generation by an LLM offers higher flexibility, but often produces unstable low-level outputs, incorrect tool usage, and hard-to-debug script errors. The proposed framework therefore uses Planner LLM together with the MCP toolkit to balance dynamic planning and reliable deployment.

The MCP toolkit is designed as a bounded deception capability space rather than an arbitrary collection of host-specific scripts. Its tool set is derived from selected MITRE Engage activities and from the environment-modification opportunities available in the AD domain. Among the three Engage goals, namely Expose, Affect, and Elicit, this work focuses on Affect activities because they most directly correspond to concrete modifications of the honeynet environment that can influence attacker progression. Expose activities mainly support observation, detection, and intelligence collection, which are handled separately by the SSH proxy and logging layer in the proposed system. Elicit activities, in contrast, aim at longer-term behavioral inducement and are therefore beyond the scope of the single-step engagement actions considered in this work.

Accordingly, each MCP tool represents a bounded defender capability for modifying the environment, such as introducing decoy identities, planting artifacts, creating credential surfaces, leaving breadcrumbs, or inducing controlled failure. Each tool exposes explicit input fields and usage constraints, allowing the Planner to assign concrete arguments to predefined capabilities rather than generate unrestricted deployment logic. Table 3 summarizes this design by group-

ing the MCP tools according to their primary deception function and mapping each group to selected MITRE Engage activities. This bounded structure makes planning more controllable, improves execution reliability, and keeps the deployed actions aligned with the engagement goals of the proposed system.

Table 3: Summary of MCP tool groups and their alignment with selected MITRE Engage activities

Tool Group	#Tools	Inspired from Engage Affect Activity						
		Lures	Personas	Artifact Diversity	Information Manipulation	Introduced Vulnerabilities	Network Manipulation	Email Manipulation
Identity and Account Exposure	3	V	V		V			
Credential Bait	3	V		V	V			V
Artifact and Data Decoys	11	V		V				
Breadcrumb and Misdirection	5	V			V		V	
Privilege Escalation Lures	2	V				V		
Service and Endpoint Lures	3	V					V	V
Controlled Failure Induction	7				V	V		V
Cross-host Movement Bait	1	V			V		V	

Planner LLM and MCP toolkit implement this design as a bounded deployment pipeline. As illustrated in Figure 5, the Planner takes engagement strategy a_i , selected tools T_i , and MCP tool specifications as input. Based on this information, it generates a structured plan P that specifies concrete tool arguments rather than low-level deployment scripts. The MCP toolkit then renders P into executable scripts S through predefined templates and deploys them to the honeynet. In this way, planning remains context-aware, while execution remains bounded by tool schemas and verified rendering logic.

The Planner stage is a constrained output problem rather than an open-ended reasoning problem. Its main requirement is stable and repeatable generation of structured tool parameters that satisfy tool schemas and remain consistent with the selected strategy and tools. For this reason, the Planner is fine-tuned instead of relying on prompt engineering alone. Supervised fine-tuning (SFT) is used to train the model to map planning context to tool arguments. Each training example pairs planner-side input, including engagement strategy, and selected tools, with the corresponding tool parameters. Following a teacher-student distillation pattern,

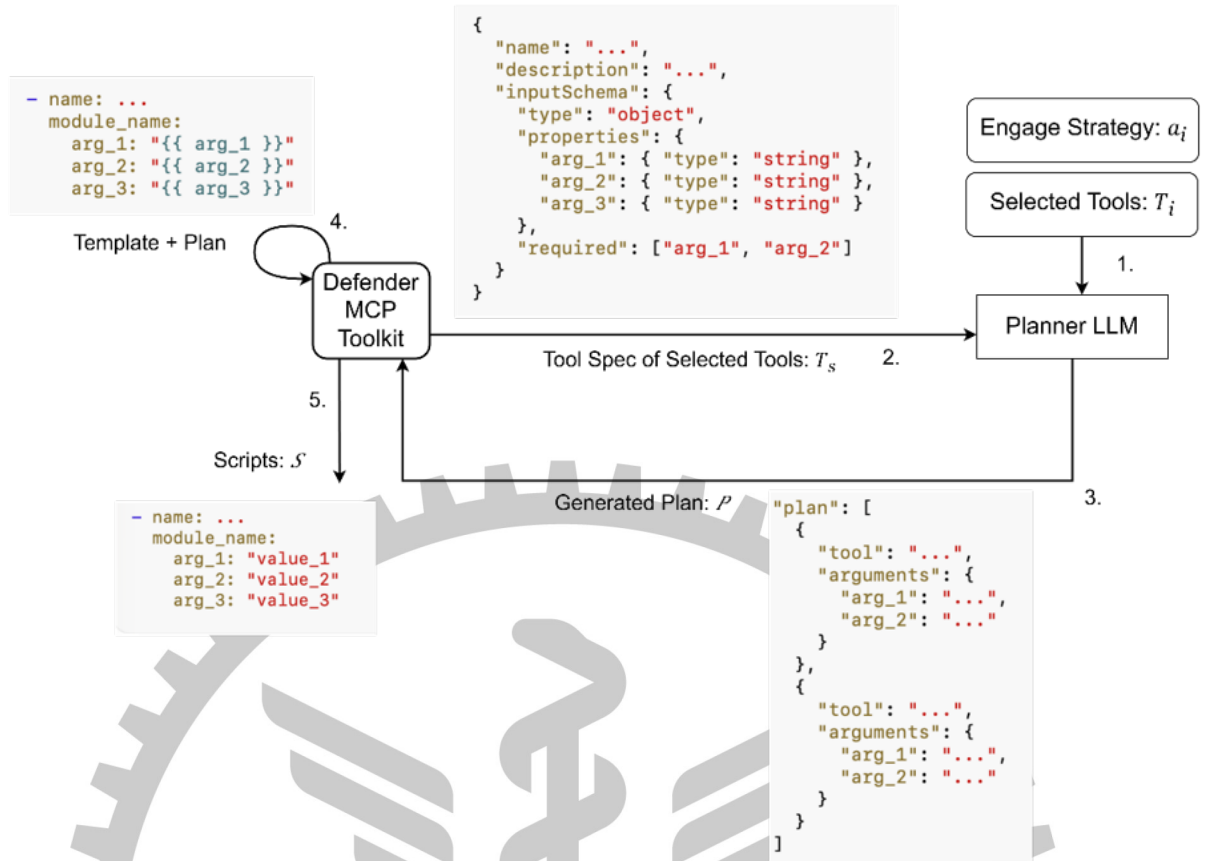


Figure 5: Tool-specification-guided plan generation and script rendering by the Planner LLM and MCP toolkit

a stronger LLM is used as a teacher model to generate planning labels, which are curated into an SFT dataset for training the smaller, locally deployed Planner model. This design makes the Planner more reliable at producing schema-compliant, complete, and deployment-ready parameters.

4.4 AD Domain Environment Design

Environment-level deception cannot rely on a single host or text-only reply generation. Later-stage attacker behaviors such as credential access, privilege escalation, and lateral movement require persistent identities, privilege relationships, service diversity, and multiple connected hosts. A single-host environment cannot provide these conditions in a realistic way, and fabricated replies alone are difficult to sustain once the attacker begins to verify files, accounts, services, or host relationships. For this reason, the proposed framework is built on a real multi-host environment in which defender modifications can create believable cross-host attack

opportunities.

As shown in Figure 6, the framework adopts an AD-based honeynet as its environment substrate. The environment includes a domain controller, Windows hosts, Linux hosts, and service hosts such as web, SQL, and mail servers. Each host role contributes a different attacker-visible surface: the domain controller provides identity and privilege structure, Windows hosts provide user and endpoint context, Linux hosts provide shell and file-system interaction surfaces, and service hosts provide application-oriented attack surfaces. As a result, the framework gains multiple places where defender tools can plant decoys, clues, credentials, and controlled obstacles that are consistent with realistic enterprise behavior.

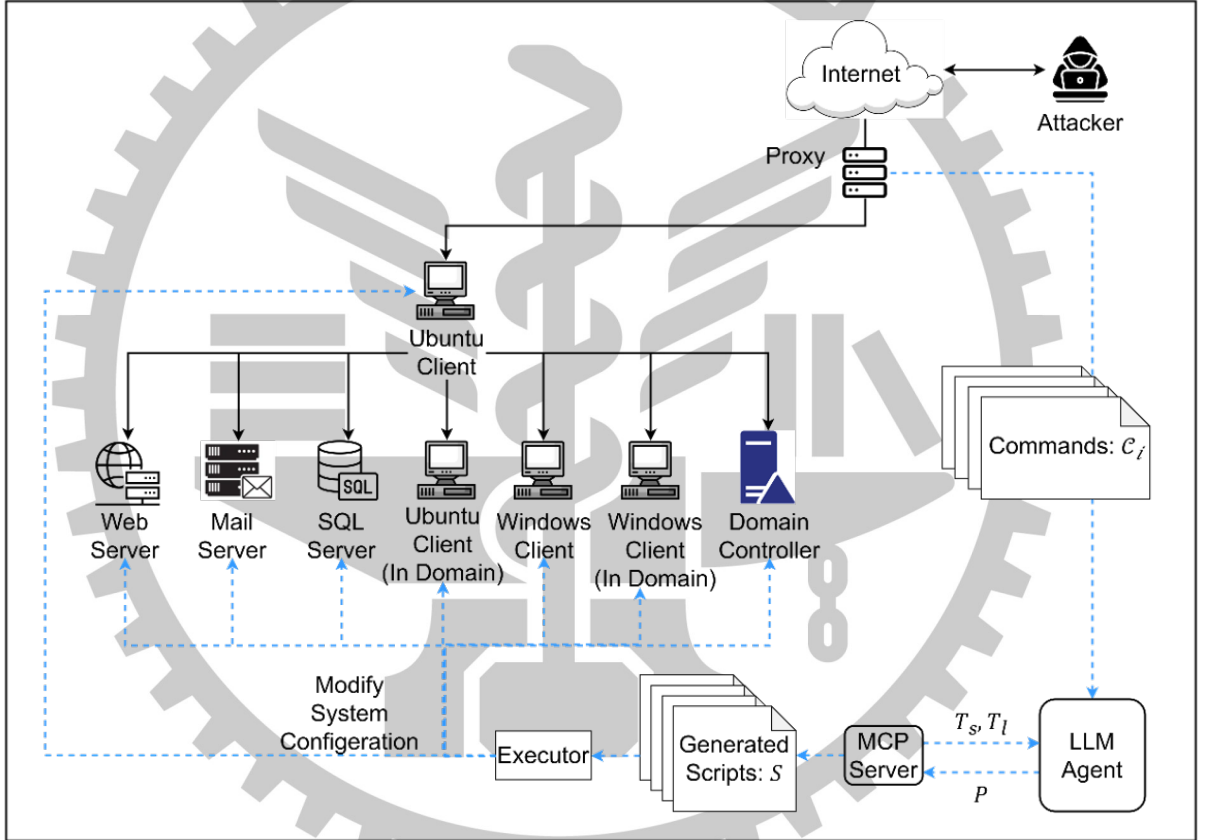


Figure 6: AD-domain-based honeynet environment with role-specific hosts and services

This AD-based multi-host design better matches real enterprise attack progression than a generic lab environment or a single-host honeypot. Because later attacker observations are grounded in actual environment changes rather than fabricated replies, cross-host deception becomes more meaningful and more sustainable over time. The AD domain therefore serves not only as the deployment target of defender actions, but also as the enterprise-like substrate

that supports deeper attack progression.



Chapter 5. Implementation

This chapter describes the implementation of the proposed Engage-Agent system. Section 5.1 presents the runtime system architecture and explains how the attacker runtime, SSH interception process, backend planning-and-execution service, and AD-domain honeynet are integrated. Section 5.2 describes the offline dataset generation and Planner fine-tuning pipeline. Section 5.3 explains how the MCP toolkit is generated, validated, and executed through Ansible-based environment modification. Finally, Section 5.4 presents the LLM-driven attacker interaction simulator used to evaluate the system under adaptive attack behavior.

5.1 System Architecture

Figure 7 shows the runtime architecture of the implemented Engage-Agent system. The implementation is divided into four runtime parts: the attacker runtime, the frontend SSH interception process, the backend planning-and-execution service, and the AD-domain honeynet. The attacker runtime maintains an interactive SSH session and generates attack commands from prior interaction history. The frontend process is a Paramiko-based MITM SSH proxy that accepts attacker traffic, extracts commands, accumulates recent command context, and invokes the Strategist LLM when the context threshold is reached. The backend service receives Strategist outputs, runs Planner-side inference, and executes MCP-based environment modifications. The AD-domain honeynet contains the live hosts and services modified by the backend and observed by the attacker. This separation reflects the actual deployment boundary of the system: SSH interception, LLM inference, backend execution, and honeynet modification are implemented as distinct runtime layers.

The attacker-facing path is implemented through one persistent SSH session between the attacker runtime and the frontend proxy. At each step, the Attacker LLM generates a command C_i from the session history, and the session holder sends it through the existing SSH connection. The MITM proxy forwards the command to the Ubuntu entry host in the AD domain while

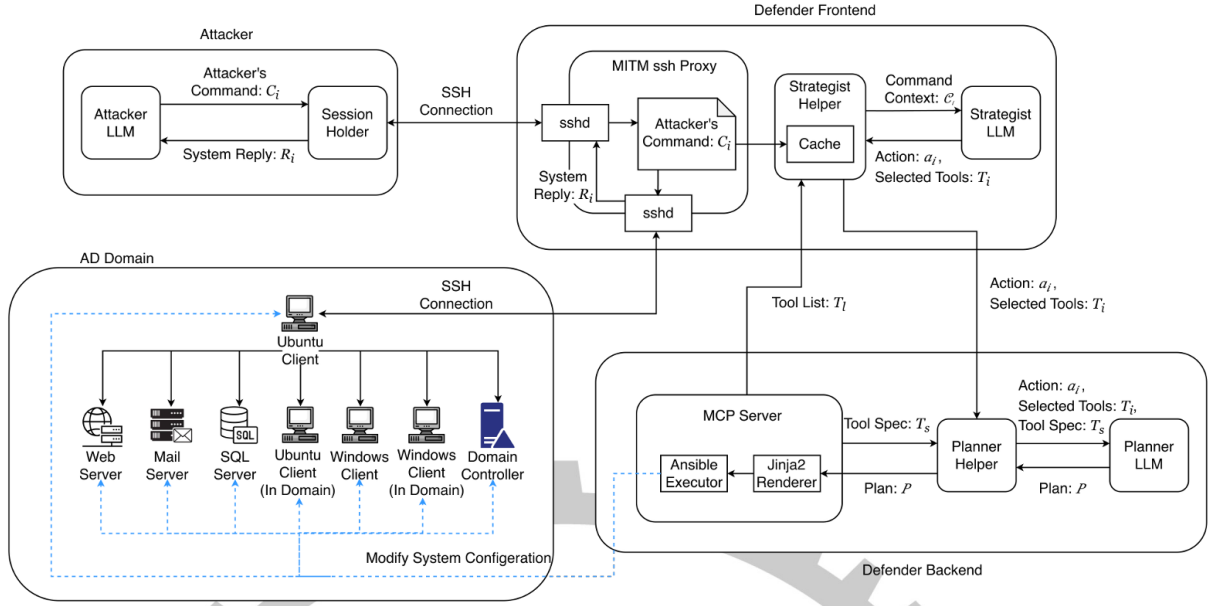


Figure 7: Runtime architecture of the implemented Engage-Agent system.

recording the command stream and session metadata. Intercepted commands are buffered as recent command context and passed to the Strategist-side process after the configured batch threshold is met. This design keeps the attacker interaction continuous while allowing the defender to analyze command context asynchronously within the same session.

The backend path starts when the frontend forwards the Strategist output, including engagement strategy a_i and selected tools T_i , to the backend API. The Planner pipeline combines these outputs with MCP tool specifications T_s and generates a structured plan P . The MCP execution module validates the plan, binds tool arguments to Jinja2-based Ansible templates, renders playbooks, and executes them through the Ansible executor. The resulting modifications are applied to the corresponding AD-domain hosts through the inventory, and their effects appear to the attacker as ordinary target-host behavior in later SSH interactions.

5.2 Dataset Generation and Fine-Tuning

Figure 8 illustrates the offline supervision pipeline used to construct the Planner fine-tuning dataset. The pipeline starts from five manually designed attack scenarios tailored to the AD-domain honeynet. Each scenario represents a common attack path over the deployed hosts and services and contains 24 attacker commands. Because the runtime system invokes the LLM after accumulating a short recent command context, each scenario is divided into three stage-level

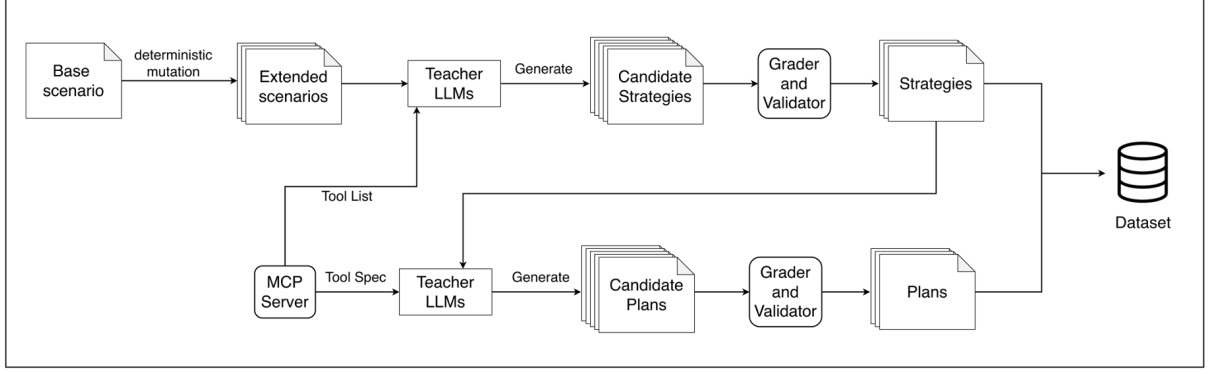


Figure 8: Offline dataset generation pipeline for Strategist supervision construction and Planner fine-tuning.

Table 4: Dataset construction scale in the offline supervision pipeline

Stage	Count	Description
Base scenarios	5	Manually designed AD-domain attack scenarios
Scenario variants	100	5 scenarios $\times$ 20 controlled variants
Stage-level samples	300	100 variants $\times$ 3 stages
Strategist candidates	1200	300 samples $\times$ 4 teacher LLMs
Selected Strategist outputs	300	One selected Strategist output per sample
Planner candidates	1200	300 Planner inputs $\times$ 4 teacher LLMs
Selected Planner outputs	300	One validated Planner output per sample
Final SFT examples	300	Planner feature-label pairs

segments of eight commands. To increase coverage while preserving the same attack logic, each base scenario is expanded into 20 controlled variants by varying environment-dependent details such as target hosts, user accounts, file paths, service names, credentials, and other scenario-specific conditions. This produces 100 scenario variants and 300 stage-level samples. These samples serve as the common input units for supervision generation and match the command granularity used during runtime inference.

Teacher-model supervision is generated in two stages. For the 300 stage-level samples, four teacher LLMs independently generate Strategist candidates, producing 1200 candidates in total. Each candidate contains attacker profiling, reasoning over the recent command context, an engagement strategy, and a shortlist of MCP tools. After structural filtering and selection, one Strategist output is retained for each stage-level sample, resulting in 300 selected Strategist outputs. The engagement strategy and selected tools in these outputs are then combined with the corresponding MCP tool specifications to form Planner-side inputs. The same four-teacher process is applied again to generate 1200 Planner candidates. These candidates contain nor-

Table 5: Planner supervised fine-tuning configuration

Item	Setting
Base model	Phi-4
Training framework	Unsloth
Fine-tuning method	QLoRA-style supervised fine-tuning
Chat template	phi-4
Max sequence length	16000
Base model loading	4-bit
Trainable parameters	LoRA adapter only
Precision	bfloat16
LoRA rank / alpha / dropout	16 / 32 / 0.05
Training batch size	2
Gradient accumulation steps	4
Learning rate	2.0×10^{-5}
Epochs	2
Optimizer	AdamW 8-bit
Learning-rate scheduler	Cosine
Warmup ratio / weight decay	0.03 / 0.01
Random seed	42
Best-model selection	Lowest validation loss
Dataset split	240 train / 30 validation / 30 test

malized plans and concrete tool arguments, and are validated against MCP schemas to ensure valid tools, complete required arguments, correct argument types, and the expected plan format. After validation and selection, one Planner output is retained for each sample. The final dataset therefore contains 300 feature-label pairs: each feature consists of the engagement strategy, selected tools, and relevant MCP tool specifications, while each label is the validated Planner output. Table 4 summarizes the sample counts at each stage of the offline supervision pipeline.

Model preparation differs across the two reasoning stages. Strategist-side supervision is used to produce structured high-level outputs and to support Planner data construction, but no Strategist SFT is performed in the implemented pipeline. Planner-side supervision is exported as the SFT target because the deployed Planner must generate schema-compliant tool parameters in a stable format. The finalized Planner dataset contains 300 samples in total and is split into 240 training samples, 30 validation samples, and 30 test samples. The Planner is then fine-tuned using a QLoRA-style supervised fine-tuning pipeline: the base Phi-4 model is loaded in 4-bit precision, while only LoRA adapter parameters are trained and saved. This setting reduces GPU memory usage while adapting the Planner to the MCP planning dataset. The main fine-tuning configuration is summarized in Table 5.

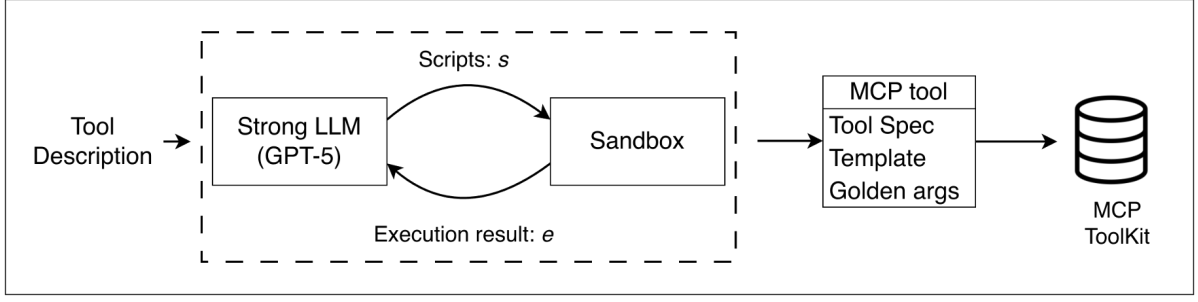


Figure 9: Semi-automated generation and validation process for MCP tools.

5.3 MCP Toolkit Generation and Execution

Figure 9 shows the build-and-validate process used to generate MCP tools for the implemented system. MCP toolkit generation begins from manually designed defender capabilities derived from selected MITRE Engage activities and the modification opportunities available on different host roles in the AD-domain honeynet. For each host type, the defender first determines what kinds of environment modifications are meaningful for attacker engagement. For example, Linux clients and service hosts can expose decoy files, breadcrumbs, and credential-like artifacts; and the domain controller can support modifications related to accounts, groups, and privilege relationships. These capabilities are designed to support engagement behaviors such as exposing decoy identities, planting artifacts, creating credential surfaces, leaving breadcrumbs, and inducing controlled failure. Each capability is then defined as one MCP tool together with a natural-language tool description. After the tool-level capability is manually specified, a strong LLM is used to generate the corresponding tool specification, Ansible template, and a set of golden arguments. The golden arguments are used to render the template and test the resulting playbook in the real AD environment. If the execution succeeds, the tool is admitted into the toolkit. If execution fails, the execution result is fed back to the LLM for debugging and revision. Therefore, MCP toolkit generation is implemented as a semi-automated build-and-validate process that reduces manual engineering effort while keeping the resulting tools executable and extensible.

After validation, each tool is stored in a bounded registry with its name, description, target host, input schema, template binding, and golden arguments. This registry gives the Planner a stable interface for generating tool calls, while allowing the MCP module to validate arguments

and render the corresponding Ansible template at runtime. As a result, every tool exposed to the Planner has a defined interface, an execution target, and a tested deployment path.

Runtime MCP execution is implemented as an in-process module inside the backend pipeline rather than as a standalone server process. Given a structured plan P , the backend invokes the MCP module with the tool name and parameter values in each plan step. The MCP module validates the arguments against the tool schema, binds them to the corresponding Jinja2-based Ansible template, and renders the final playbook. The playbook is then executed through a subprocess call to the Ansible command-line tool, which connects to the target hosts through SSH or WinRM and applies the requested modifications. This path keeps runtime execution traceable from Planner-generated tool calls to concrete changes in the AD-domain honeynet.

5.4 Attacker Interaction Simulation

Attacker simulation is implemented as an LLM-driven interactive command loop rather than a fixed replay script. At each step, the attacker model receives recent session history, including previous commands and observed replies, and generates the next command with structured metadata describing the intended action. The command is executed through a persistent SSH session connected to the defender-side proxy, allowing the simulated attacker to interact with the honeynet as a continuous shell session. Because multiple commands share the same shell stream, sentinel markers are inserted around each command execution to recover the corresponding output segment and return code. This design allows attacker behavior to evolve from prior observations and react to environment changes introduced during the session.

The Attacker LLM is guided by a seven-stage attack progression: discovery, collection, credential access, privilege escalation, lateral movement, exfiltration, and impact. At each step, the model selects the next attack direction from the current session history and then generates a command for that direction. To make the simulation resemble a credible attacker, the prompt requires the model to rely on evidence discovered in the environment, recognize repeated unrealistic responses as possible honeypot signals, pursue lateral movement when useful credentials or host information are found, and treat exfiltration or impact as late-stage objectives. For stages that are difficult to execute reliably through free-form generation, such as privilege escalation, lateral movement, exfiltration, and impact, the prompt provides reference command templates.

These templates guide command construction without fixing a predetermined attack path, preserving open-ended behavior while reducing unnecessary trial and error.

